Helicopter LAB report

TTK4135 - Optimization and regulation

Student 564168
Student 539162

March 2, 2026

Department of Engineering Cybernetics

Contents

1 Introduction **1**

2 System description **1**

3 Equations of motion **1**

4 10.2 - Optimal Control of Pitch/Travel without Feedback **2**

4.1 The continuous model 2
4.2 The discretized model 2
4.3 The open loop optimization problem 3
4.4 The weights of the optimization problem 4
4.5 MATLAB and Simulink 5

5 10.3 - Optimal Control of Pitch/Travel with Feedback (LQ) **9**

5.1 LQ controller 9
5.1.1 LQ/LQR 9
5.1.2 Q and R matrix 9
5.1.3 Our weights 10
5.2 Model Predictive Control 10
5.2.1 Pros and Cons 10
5.2.2 The figure changes 10
5.3 Experimental results 11
5.4 MATLAB and Simulink 14

6 10.4 - Optimal Control of Pitch/Travel and Elevation with Feedback **16**

6.1 The continuous model 16
6.2 The discretized model 16
6.3 Experimental results 16
6.4 Decoupled model 20
6.5 MATLAB and Simulink 20
6.5.1 Code from section 6 20
6.6 Simulink block diagrams 24
6.6.1 Simulink from section 6 24

7 Reflection **25**

References **27**

1 Introduction

The goal of this project was to provide the students an understanding in how MPC (Model Predictive Control) can be used in solving an optimization problem where optimal control with and without feedback is the expected result. The lab project was spread out across multiple days, and the added plots, code and SIMULINK-block diagrams aims to show the work put into this task.

In the two following chapters, 2 and 3 respectively, the physical set up of the lab will be described, as provided to us in the lab description and in the lab areas where the physical model of the helicopters were implemented.

In addition to polishing the report based on feedback, a new sub chapter was introduced. In "Reflection" we aimed to add our thoughts on decisions in the correction of this project and report. Please have this chapter in mind when reviewing this final iteration.

2 System description

The following system consists of three degrees of motion. The Matlab project code is linked to the physical system through a QUARC real-time executable.

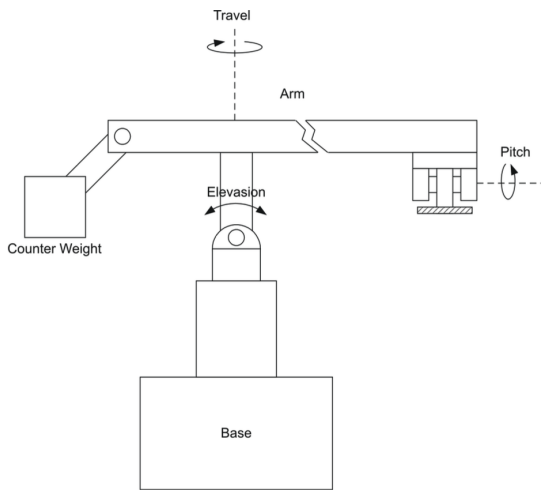


Figure 1: Cross section representation of the model provided by TTK4145 [1].

3 Equations of motion

These are the equations of motion that serve as the basis when deriving the state-space models in continuous time later in the report. All the equations are provided in the lab document [1].

$$\ddot{e} + K_3 K_{ed} \dot{e} + K_3 K_{ep} e = K_3 K_{ep} e_c \tag{1}$$

$$\ddot{p} + K_1 K_{pd} \dot{p} + K_1 K_{pp} p = K_1 K_{pp} p_c \tag{2}$$

$$\dot{\lambda} = r \tag{3}$$

$$\dot{r} = -K_2 p \tag{4}$$

4 10.2 - Optimal Control of Pitch/Travel without Feedback

For the initial part of this lab-project, we will disregard the elevation aspect in the elevator completely when solving for an optimal trajectory x^* with a corresponding actuation input u^* for each of the states in the helicopter model. To do this, we must solve the problem as a quadratic optimization. As per the task definition, a 5 second input delay through zero-padding was introduced. This can be seen as the helicopter holds the tracking until the 5 seconds has passed. The first task is to formulate the equations of motion on state space form.

4.1 The continuous model

The model is first depicted on continuous form, based on the equations of motion. The state variables we introduce are the following $x = \begin{bmatrix}\lambda & r & p & \dot{p}\end{bmatrix}^T$ and $u = p_c$, where λ represents travel, r : travel rate, p : pitch and $\dot{p}$: pitch rate. Writing this on the form $\dot{x} = A_c x + B_c u$ using the predefined eqms we get the following continuous state space system:

$$\dot{x} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -K_2 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -K_1 K_{pp} & -K_a K_{pd} \end{bmatrix} \begin{bmatrix} \lambda \\ r \\ p \\ \dot{p} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ K_1 K_{pp} \end{bmatrix} u(p_c) \tag{5}$$

4.2 The discretized model

Trying to discretize Equation 5 using forward the Euler method, the formula can be recalled from Calculus, and the derivation can look like this:

$$\frac{x_{k+1} - x_k}{\Delta t} = \dot{x} \tag{6}$$

$$\frac{x_{k+1} - x_k}{\Delta t} = A_c x_k + B_c u_k \tag{7}$$

$$x_{k+1} - x_k = (A_c x_k + B_c u_k)\Delta t \tag{8}$$

$$x_{k+1} = (A_c x_k + B_c u_k)\Delta t - x_k \tag{9}$$

$$x_{k+1} = (I - \Delta t A_c) x_t + \Delta t B_c u_k \tag{10}$$

Now inserting for the values we defined in equation 5 we get that the discretized model of the state space system becomes:

$$\dot{x} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & -\Delta t K_2 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & -K_1 K_{pp} & 1 - K_a K_{pd} \end{bmatrix} \begin{bmatrix} \lambda \\ r \\ p \\ \dot{p} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ \Delta t K_1 K_{pp} \end{bmatrix} u(p_c) \tag{11}$$

By the problem description we define $\Delta t = 0.25$.

4.3 The open loop optimization problem

We need to implement the constraint $|p_k| \leq \frac{60\pi}{360}$ on the pitch angle as to minimize the range of the pitch to the actual range of the helicopter model. To calculate the optimal trajectory to move the helicopter from its initial position $x = \begin{bmatrix} \lambda_0 & 0 & 0 & 0 \end{bmatrix}^T$ to its reference position $x_f = \begin{bmatrix} \lambda_f & 0 & 0 & 0 \end{bmatrix}^T$ the cost function has to be minimized, meaning formulating the following equation as an optimization problem and then using the "quadprog"-function in MATLAB.

$$\phi = \sum_{i=1}^{N} (\lambda_{i+1} - \lambda_t)^2 + q_1 p_{ci}^2 + q_2 e_{ci}^2, \quad q \geq 0. \tag{12}$$

An optimization problem is formulated using the fact that we have 4 states and N = 100 time-steps

$$\min_{z \in R^2} f(z) = \frac{1}{2} z^T G z, \quad A_{eq} z = b_{eq} \tag{13}$$

$$A_{eq} = \begin{bmatrix} I & & & | & -B & & \\ -A & I & : & | & & -B & : \\ & .. & & | & & .. & \\ & -A & I & | & & & -B \end{bmatrix} \tag{14}$$

Using the same strategy we can generate the G matrix, and will then after defining the upper and lower bounds be able to solve the optimization-problem

$$G = \begin{bmatrix} Q & .. & .. & .. & 0 \\ : & Q & : & & : \\ : & .. & & .. & : \\ : & & : & q & : \\ 0 & .. & .. & .. & q \end{bmatrix} \tag{15}$$

The QP can then pe solved using the following code snippet

```
Q1 = zeros(mx,mx);
Q1(1,1) = 1;                                % Weight on state x1
Q1(2,2) = 1;                                % Weight on state x2
Q1(3,3) = 1;                                % Weight on state x3
Q1(4,4) = 1;                                % Weight on state x4
q = 12;                                     % Weight on input
G = gen_q(Q1, q, M, N);                     % Generate Q, hint: gen_q
c = zeros(500,1);

%% Generate system matrixes for linear model
Aeq = gen_aeq(A, B, N, mx, mu);          % Generate A
beq = zeros(400,1);                         % Generate b
beq(1:4) = A1*x0;
```

```
%% Solve QP problem with linear model
tic
[z,lambda] = quadprog(G,c,[],[], Aeq, beq, vlb, vub);
t1=toc;
```

4.4 The weights of the optimization problem

For this task, we were to try different weighting values of q in order to tune the system. The values that were tested were $q = 0.12$, $q = 1.2$ and $q = 12$. The initial position of the elevator were set to $\pi = 180$, a

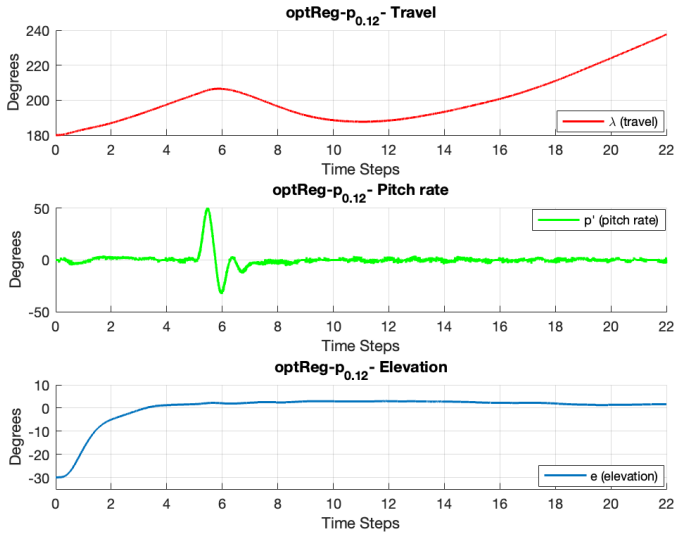


Figure 2: q = 0.12.

With a small q like $q = 0.12$ one can see that the limitation on the actuation $u_k = p_k$ is minuscule, causing the system to apply massive correcting force in order to counteract the deviation from the reference point, as seen in figure 2. With no feedback control, the elevator has no information about its states after the corrective force has been added. Therefore, the helicopter gets this steady state travel, with no ability to correct its positioning.

Increasing the weight tenfold to $q = 1.2$ gives us the result as seen in figure 3. The actuation which can most clearly be seen in pitch rate is a fraction smaller, which could be a good sign, considering the massive input provided. However, with a stricter weighting on the input through q, the initial position correction is less mitigated as the input is weaker than before.

Now further decreasing the allowed u_k by increasing the weight $q = 12$ we notice that the oscillatory changes in the pitch rate is drastically reduced, and that the overshooting correction in the pitch is also reduced. Nevertheless, with a q this high the effect of the input actuation is to low to ever correct the travel positioning to the reference value in open loop configuration.

Considering all values of Q in the same plot can be seen in Figure 5.

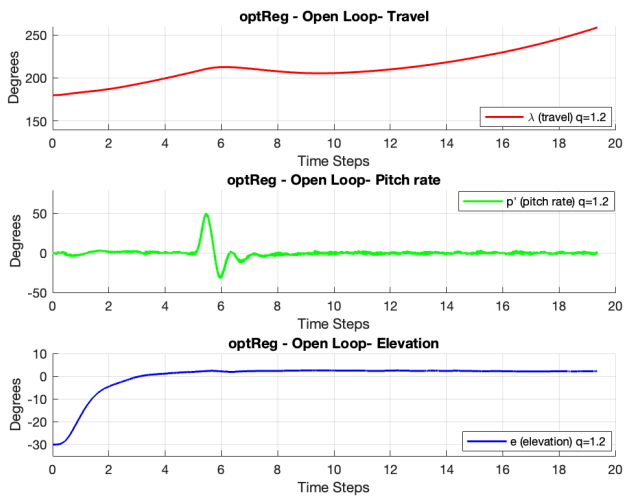


Figure 3: q = 1.2

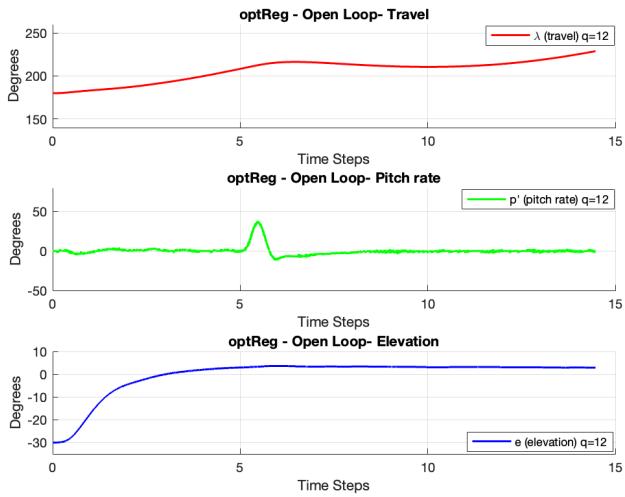


Figure 4: q = 12

4.5 MATLAB and Simulink

```
    % Discrete time system model. x = [lambda r p p_dot]'
delta_t = 0.25; % sampling time
A1 = [
    1, delta_t, 0, 0;
    0, 1, -delta_t*K_2, 0;
    0, 0, 1, delta_t;
    0, 0, -K_1*K_pp * delta_t, 1 - K_1*K_pd * delta_t];
B1 = [
    0;
    0;
    0;
    K_1*K_pp * delta_t
```

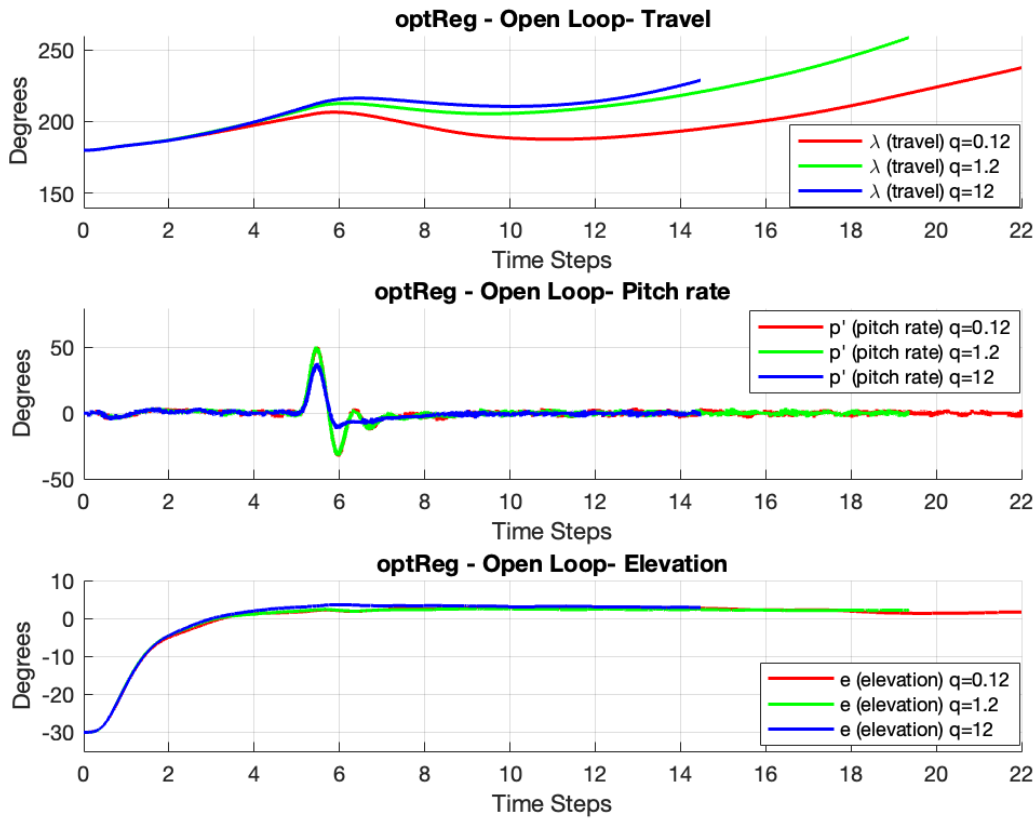


Figure 5: Caption

```
    ];

% Number of states and inputs
mx = size(A1,2); % Number of states (number of columns in A)
mu = size(B1,2); % Number of inputs(number of columns in B)

% Initial values
x1_0 = pi;                      % Lambda
x2_0 = 0;                       % r
x3_0 = 0;                       % p
x4_0 = 0;                       % p_dot
x0 = [x1_0 x2_0 x3_0 x4_0]';    % Initial values

% Time horizon and initialization
N  = 100;                       % Time horizon for states
M  = N;                         % Time horizon for inputs
z  = zeros(N*mx+M*mu,1);        % Initialize z for the whole
   horizon
z0 = z;                         % Initial value for optimization
```

```
% Bounds
ul          = 0;                    % Lower bound on control
uu          = 1;                    % Upper bound on control

xl      = -Inf*ones(mx,1);      % Lower bound on states (no bound)
xu      = Inf*ones(mx,1);       % Upper bound on states (no bound)
xl(3)   = ul;                   % Lower bound on state x3
xu(3)   = uu;                   % Upper bound on state x3

% Generate constraints on measurements and inputs
[vlb,vub]       = gen_constraints(N,M,xl,xu,ul,uu); % hint:
   gen_constraints
vlb(N*mx+M*mu)  = 0;            % We want the last input to be
   zero
vub(N*mx+M*mu)  = 0;            % We want the last input to be
   zero

% Generate the matrix Q and the vector c (objecitve function
   weights in the QP problem)
Q1 = zeros(mx,mx);
Q1(1,1) = 1;                    % Weight on state x1
Q1(2,2) = 1;                    % Weight on state x2
Q1(3,3) = 1;                    % Weight on state x3
Q1(4,4) = 1;                    % Weight on state x4
P1 = 12;                        % Weight on input
Q = gen_q(Q1, P1, M, N);        % Generate Q, hint: gen_q
c = zeros(500,1);

%% Generate system matrixes for linear model
Aeq = gen_aeq(A1, B1, N, mx, mu);          % Generate A
beq = zeros(400,1);                        % Generate b
beq(1:4) = A1*x0;

%% Solve QP problem with linear model
tic
[z,lambda] = quadprog(Q,c,[],[], Aeq, beq, vlb, vub);
t1=toc;
```

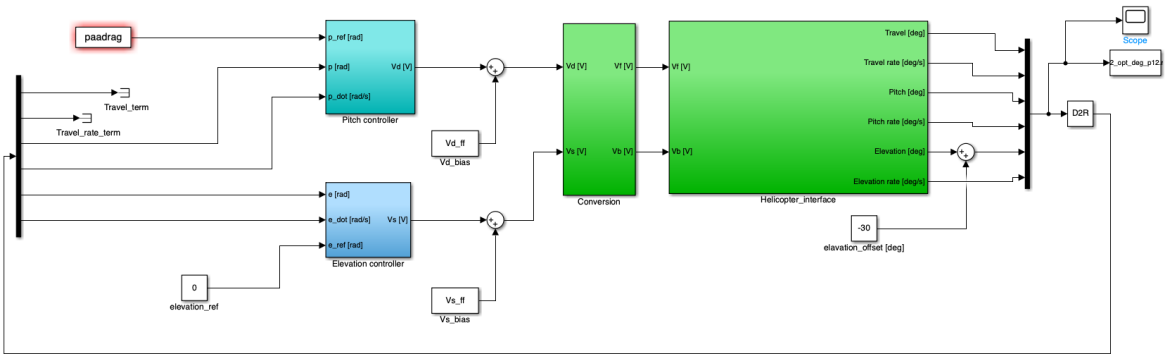


Figure 6: Simulink of the helicopter system from task 10.2.

5 10.3 - Optimal Control of Pitch/Travel with Feedback (LQ)

5.1 LQ controller

5.1.1 LQ/LQR

An LQ controller is a type of state feedback controller that is used to stabilize a linear system over time. LQ or LQR linear quadratic regulator also tries to minimize a cost function as the system stabilizes.

For an LQ we assume that the system can be represented as:

$$x_{k+1} = Ax_k + Bu_k$$

The goal of the controller is to compute the required input u_k that drive the system to the desired state x_k^* while minimizing the cost function, written as;

$$J = \sum_{k=0}^{\infty} \Delta x_k^\top Q \Delta x_k + \Delta u_k^\top R \Delta u_k$$

here,

- Q is a symmetric positive semi-definit matrix
- R is a symetric positive definit matrix
- Δx_k is the deviation from the optimal state
- Δu_k is the deviation from the optimal control input

Using the Riccati equation we can create a feedback gain matrix K used in the optimal control law;

$$u_k = u_k^* - K(x_k - x_k^*)$$

5.1.2 Q and R matrix

The matrices Q and R relate to the cost function regarding weights. Each matrix represent a "configuration" of the controller, different weights of these matrices yields different attributes of the controller.

- **Q matrix**

  The purpose of the Q matrix is to penalize deviation in the states x_k from the desired trajectory x_k^*. Increasing the values in the Q matrix yields a controller that try to follow the desired trajectory strictly, and bring the state variable closer to their reference. Increasing this matrix may result in a controller that act aggressively and suboptimal on small variations of the system states.

- **R matrix**

  Unlike Q the R matrix constrict the controller. The weight of R determines the penalization of control input, how conservative the controller shall be. Higher values in the R matrix yields a smoother controller. On the other side low values in R result in an aggressive controller.

5.1.3 Our weights

Upon running the simulation, we tested different weights on both matrices. We started running the simulation with some generic values like Q equal to the identity matrix and $R = 1$. The plot in figure 7 shows the trajectory. We tried changing the controllers sensitivity to the different system states, and the penalization of control input. After some adjustments to the weight our best outcome was when,

$$Q = \begin{bmatrix} 5 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R = 0.5$$

The evolution of the system states during the simulation using these weights can be seen in figure 9. Using these weights the controller gained the least overshoot, and we got a relativlty smooth control input. Since the controller only has access to changing the pitch, changes to R does not directly impact the other states drastically. With this configuration of Q and R we got some gentle correction and we reduced the overshoot on the travel. A smoother control signal benefits as this causes the system to be more stable, less wear on the equipment and more human friendly motion. Due to errors in the lowerbounds of the control input, the plots discusse in this chapter may not display each controller configuration's true potential, however this will be addressed more clearly later in the report.

5.2 Model Predictive Control

To implement MPC on this problem, we could write a MatLab script that calculates the optimal control input. This script would be runned at every timestep during the simulation, updating the optimal control input, and preforming the first control input and then repeating at the next time step.

5.2.1 Pros and Cons

Prediction and LQ is simple to implement after the initial planning. This solution is also computational easy, since it only requires matrix multiplication, making this an effective way of solving the problem computational wise. However this solution comes with some disadvantages. If there are changes to the system states or the enviroment, this solution would not adapt. If deviations appear this controller will only correct it locally with the LQ gain. Lastly a predictive + LQ controller assumes precomputated trajectory is allways ideal. On the contrary MPC has the ability to adapt to changes and disturbances in real time. MPC has the ability to handle changes in the constraint, and the continuous computation causes this to be favorable for long-term tracking. However the continuous computation introduces some flaws as well. Computing a new optimal control input at each time step requires high computational load, and is more complex to debug and tune. We also need to implement a real-time solver which has to be quick enough to preform the optimal control input.

5.2.2 The figure changes

If we implemented MPC there would be some changes to the figure. Both the block named "Model-based optimization" and the "LQR" block would be merged into a new block called

"MPC". This block would use the current state of the system to calculate the optimal control input, and would preform the first input control at each time step.

5.3 Experimental results

The first simulation we ran the weight of both Q and R had values of 1, shown in figure 7. These basic weights offer a great starting point as the controller is not too aggresive yet not too constricted either. The next plot was from the second iteration, Q was altered. With higher values in Q the controller becomes more aggressive reaching null point with more haste than the first iteration. The downside is the deviation in elevation, since this controller have no direct control of elevation, we get a significant "dip" in elevation, seen in figure 8.

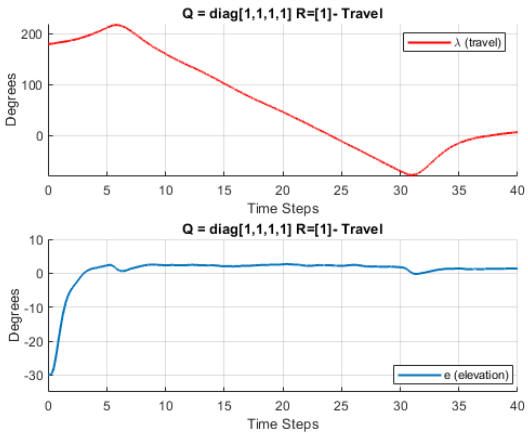


Figure 7: Plot of the system states with Q as the identity matrix and $R = 1$.

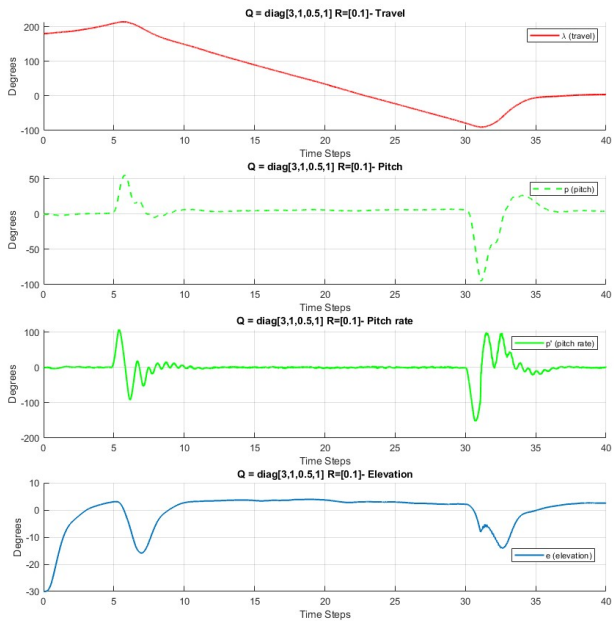


Figure 8: Plot of the system states with Q as $\begin{bmatrix} 3 & 1 & 0.5 & 0.1 \end{bmatrix}$ as the diagonal and $R = 0.5$.

In figure 9 we experimented with different weight on the different system states. This configuration caused the controller to be less aggressive and overshoot less than the iteration before. This is due to the low value of R at the same time as increased focus on the weight of travel and decreased weight of pitch in Q. Next iteration we tuned the Q matrix further yields the plot in figure 10. Here the controller is penalized more than the previous controller and respond more aggresivly to the optimal travel and pitch and care less about the travel rate and pitch rate. Closely inspection of the plot shows that the pitch rate is higher in this plot than the previous.

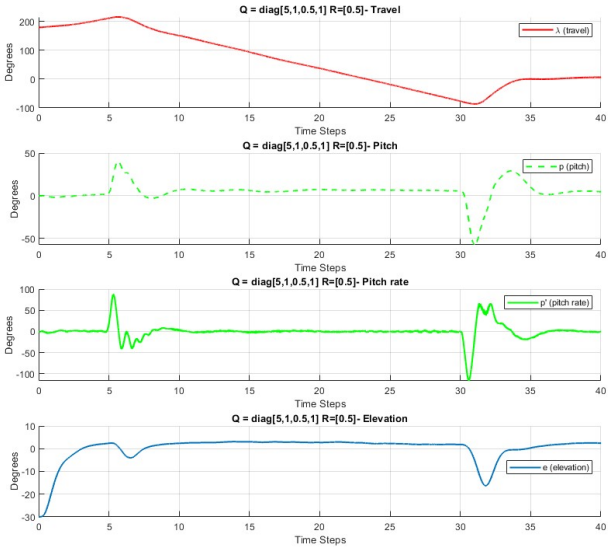


Figure 9: System states with $R = 0.5$ and Q a diagonal matrix with the vector $\begin{bmatrix} 5 & 1 & 0.5 & 1 \end{bmatrix}$ on the diagonal.

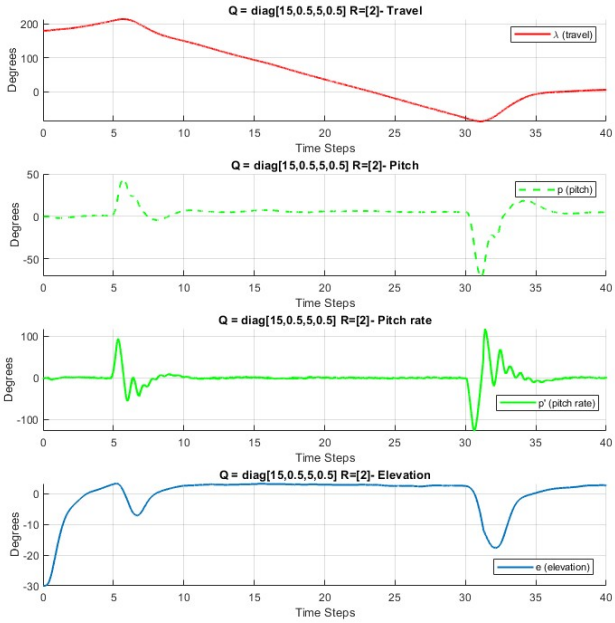


Figure 10: System states with $R = 2$ and Q a diagonal matrix with the vector $\begin{bmatrix} 15 & 0.5 & 5 & 0.5 \end{bmatrix}$ on the diagonal.

The weights in the last plot, figure 11, is the same as figure 10 with the exception to R. The reduction in control input penalization gives us more oscillation in the pitch rate and lower "dips" in the elevation. This weights is arguably the worst, as the controller be comes to aggressive. Due to the similarity of Q in the last two iterations, we can see they have similar, almost identical behaviour. Both pitch and pitch rate has the same form, with different values.

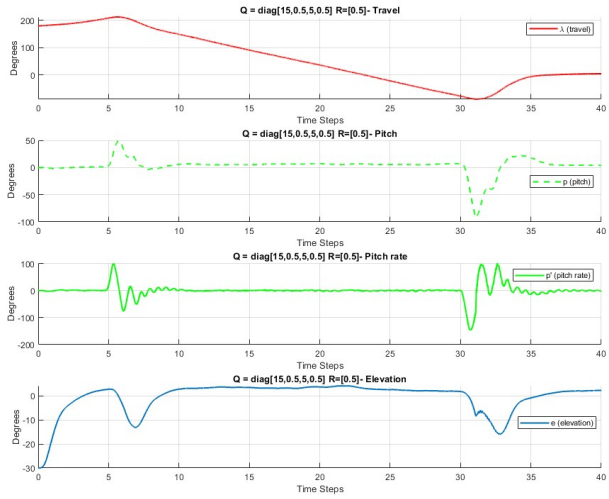


Figure 11: System states with $R = 0.5$ and Q with $\begin{bmatrix} 15 & 0.5 & 5 & 0.5 \end{bmatrix}$ as the diagonal.

Printouts of data from relevant experiments (plots). Discussion and analysis of the results. Answer 10.3.2.5 here.

5.4 MATLAB and Simulink

Code and diagrams go here

```
%% Initialization and model definition
"../init_files_2021_v2/init.m"; % Change this to the init file
   corresponding to your helicopter

delta_t = 0.25; % sampling time

%System matrices
A1 = [
    1, delta_t, 0, 0;
    0, 1, -delta_t*K_2, 0;
    0, 0, 1, delta_t;
    0, 0, -K_1*K_pp * delta_t, 1 - K_1*K_pd * delta_t];
B1 = [
    0;
    0;
    0;
    K_1*K_pp * delta_t
    ];



% Time horizon and initialization
N  = 100;                                   % Time horizon for
   states
M  = N;                                  % Time horizon for inputs
z  = zeros(N*mx+M*mu,1);                 % Initialize z for the
   whole horizon
z0 = z;



diagonal_1 = [10,.1,.1,.1];
%Defining components in LQ-controller
Q = diag(diagonal_1);
R = diag(10);
disp(Q)
disp(R)
%Solving for optimal gain matrix K
[K,S,P] = dlqr(A1, B1, Q, R);
```

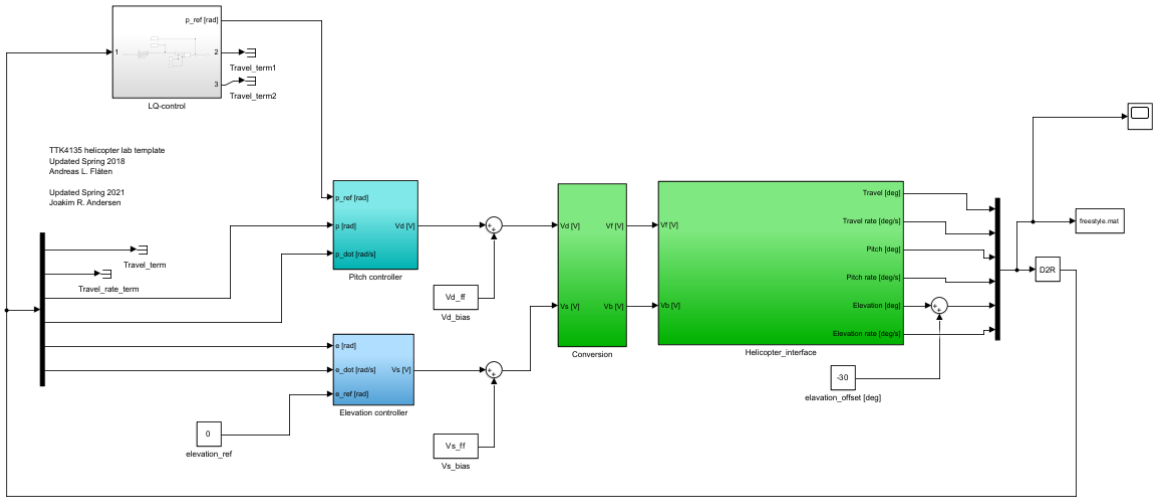


Figure 12: Simulink of the heli system from task 10.3.

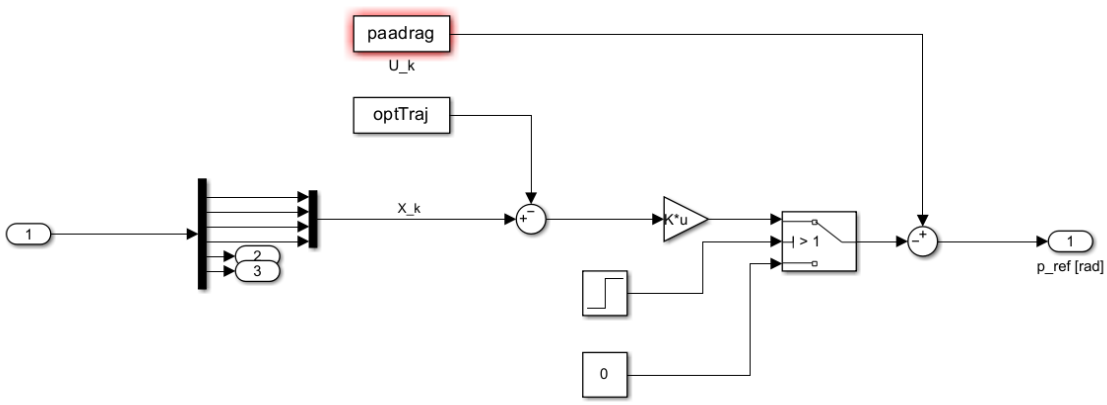


Figure 13: Simulink of the controller used in the heli system from task 10.3.

6 10.4 - Optimal Control of Pitch/Travel and Elevation with Feedback

6.1 The continuous model

We write the continuous state space model in the form $\dot{x} = Ax + Bu$ as in 5, where in x we introduce the two new state variables elevation e and elevation rate $\dot{e}$, so that $x = \begin{bmatrix}\lambda & r & p & \dot{p} & e & \dot{e}\end{bmatrix}^T$.

This new state space vector is compared to the given equations of motion provided in the task description 3, and the matrix variables were then derived to provide the following continuous model:

$$\begin{bmatrix}\dot{\lambda}\\ \dot{r}\\ \dot{p}\\ \ddot{p}\\ \dot{e}\\ \ddot{e}\end{bmatrix} = \begin{bmatrix}0 & 1 & 0 & 0 & 0 & 0\\ 0 & 0 & -K_2 & 0 & 0 & 0\\ 0 & 0 & 0 & 0 & 0 & 0\\ 0 & 0 & -K_1K_{pp} & -K_1K_{pd} & 0 & 0\\ 0 & 0 & 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0 & -K_3Kep & -K_3Ked\end{bmatrix}\begin{bmatrix}\lambda\\ r\\ p\\ \dot{p}\\ e\\ \dot{e}\end{bmatrix} + \begin{bmatrix}0 & 0\\ 0 & 0\\ 0 & 0\\ K_1K_{pp} & 0\\ 0 & 0\\ 0 & K_3K_{ep}\end{bmatrix}\begin{bmatrix}p_c\\ e_c\end{bmatrix} \tag{16}$$

6.2 The discretized model

The model was then discretized using the forward Euler method as in 5, which for state space systems is defined as follows: $x_{k+1} = (I - \Delta t A_c)x_t + \Delta t B_c u_k$. A_c and B_c represents the continuous matrices found in the previous subsection 6.1. Writing this in Matlab for further use was done as shown below, and the mathematical dicection follows after:

```
delta_t = 0.25; % sampling time
...
A_disc = eye(6) + delta_t * A;
B_disc = delta_t * B;
```

$$\begin{bmatrix}\dot{\lambda}\\ \dot{r}\\ \dot{p}\\ \ddot{p}\\ \dot{e}\\ \ddot{e}\end{bmatrix} = \begin{bmatrix}1 & \Delta t & 0 & 0 & 0 & 0\\ 0 & 1 & -\Delta t K_2 & 0 & 0 & 0\\ 0 & 0 & 1 & \Delta t & 0 & 0\\ 0 & 0 & -\Delta t K_1 K_{pp} & 1 - \Delta t K_1 K_{pd} & 0 & 0\\ 0 & 0 & 0 & 0 & 1 & \Delta t\\ 0 & 0 & 0 & 0 & -\Delta t K_3 Kep & -\Delta t K_3 Ked\end{bmatrix}\begin{bmatrix}\lambda\\ r\\ p\\ \dot{p}\\ e\\ \dot{e}\end{bmatrix} + \begin{bmatrix}0 & 0\\ 0 & 0\\ 0 & 0\\ \Delta t K_1 K_{pp} & 0\\ 0 & 0\\ 0 & \Delta t K_3 K_{ep}\end{bmatrix}\begin{bmatrix}p_c\\ e_c\end{bmatrix} \tag{17}$$

6.3 Experimental results

As for the results of implementing these two new states in elevation (e) and elevation rate ($\dot{e}$), adding a non-linear inequality constraint, minimizing the provided objective function ϕ, there were a lot of variables to tune. After the new state variables were implemented, the objective function ϕ was next, but not without the constraint of nonlinear inequality. They were defined as follows:

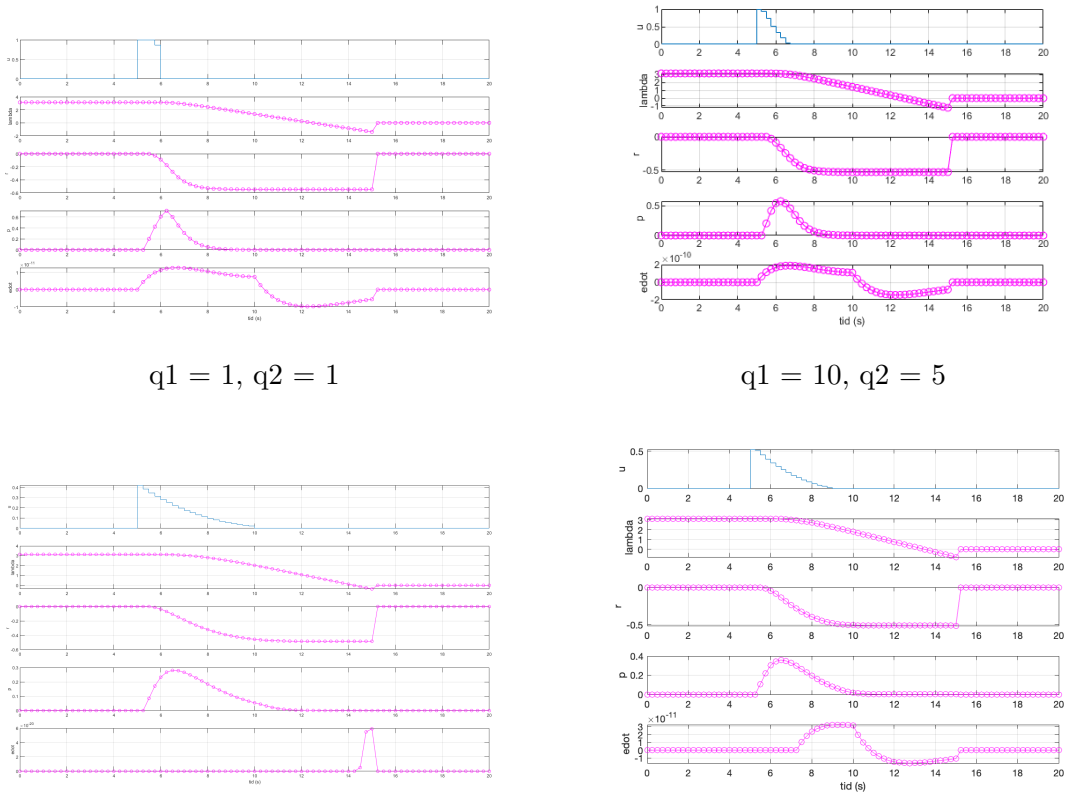
Objective function ϕ:

$$\phi = \sum_{i=1}^{N} (\lambda_{i+1} - \lambda_t)^2 + q_1 p_{ci}^2 + q_2 e_{ci}^2 \tag{18}$$

Non-linearity-constraint:

$$c(x_k) = \alpha \exp(-\beta(\lambda_k - \lambda_t)^2) - e_k \leq 0 \tag{19}$$

The values for α, β and λ_t were defined as 0.2, 20 and $\frac{2\pi}{3}$ respectively. Tuning for different values of q_1 and q_2 can be seen in 14 below, but we found the most promising response that balanced stability and robustness with efficiency were $q_1 = 10$ and $q_2 = 5$



q1 = 1, q2 = 1

q1 = 10, q2 = 5

q1 = 50, q2 = 5

q1 = 50, q2 = 20

Figure 14: Testing the effect of values q_1 and q_2.

At the lab, what remained was solving the optimization problem with the newly introduced non-linear constraint, before tuning the Q and R matrices in the LQR controller. To do this we had to replace the "quadprog" function for solving as we did for linear constraints in the earlier assignments. Instead the function "fmincon" was used, which works as a Sequential Quadratic Programming-based algorithm. The strategy for tuning this regulator was to iteratively add more or less weight to the parameters we deemed most influential. The states shown in each of the plots are travel, pitch, pitch-rate and elevation as they were the necessary parameters to graphically represent the physical helicopter properly.

The first plot 15 shows the output when all of the states were equally weighted. The weights of the R matrix were set to 0.5 and 0.5, equally across all tests. We see that the travel and pitch follows the calculated solution closely, but the pitch rate is rather aggressive.

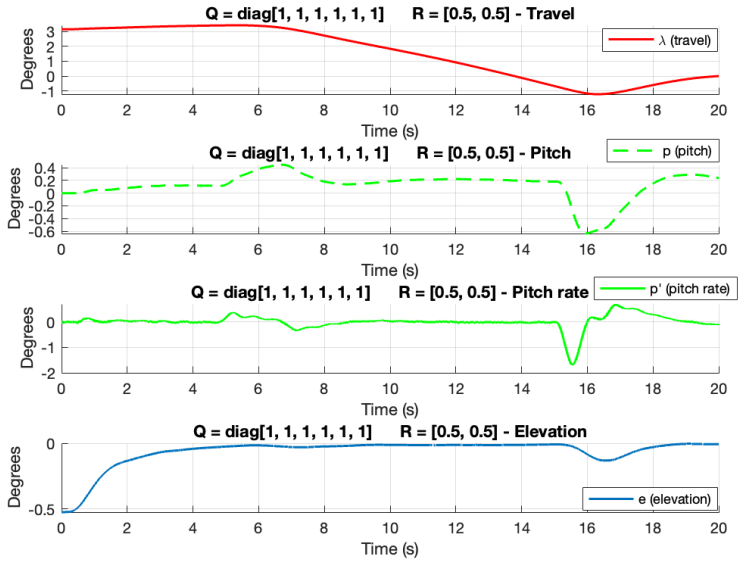


Figure 15: Output of Q = diag$(1, 1, 1, 1, 1, 1)$ and R = diag$(0.5, 0.5)$

Building on 15 for the second plot, the overshoot in travel proved unwanted, and as a result the Q-values in 16 was an attempt to mitigate this travel. However, reducing the penalty of all other states drove the system into an unstable behavior. This can be seen in the pitch and elevation subplots, as the bounces shows ground contact.

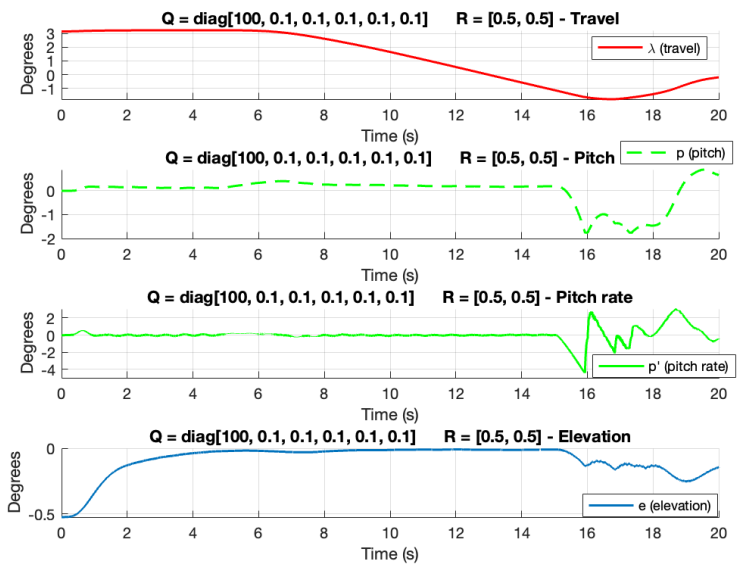


Figure 16: Q = diag$(100, 0.1, 0.1, 0.1, 0.1, 0.1)$ and R = diag$(0.5, 0.5)$

Further development of the tuning of the LQR-regulator, the deviation of elevation proved important, as without penalizing elevation deviation, the pitch regulation fo the helicopter paid no attention to elevation drop as seen in 16. When increasing the weighting of elevation and elevation rate as in 17, the system behaved better but yet not satisfactory.

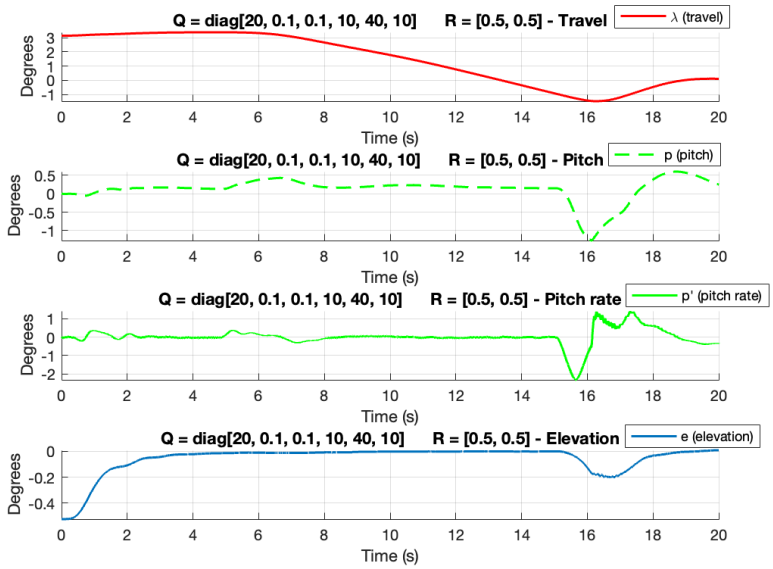


Figure 17: $\mathrm{Q} = \mathrm{diag}(20, 0.1, 0.1, 10, 40, 10)$ and $\mathrm{R} = \mathrm{diag}(0.5, 0.5)$

With some further tuning, the following values in 18 ended up being the preferred ones for the system. Even when increasing the prediction horizon N, the overshoot in travel would not yield, but with the current tuning the helicopter showed smooth transitions between the reference states.

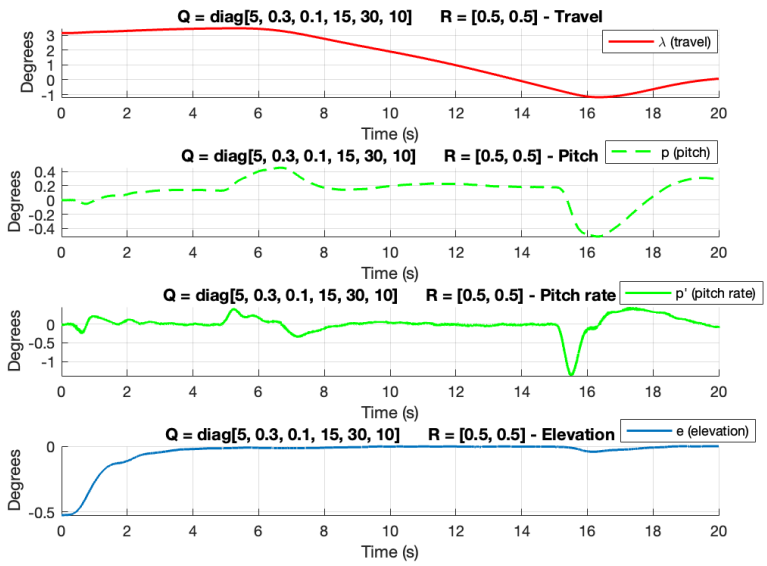


Figure 18: $\mathrm{Q} = \mathrm{diag}(5, 0.3, 0.1, 15, 30, 10)$ and $\mathrm{R} = \mathrm{diag}(0.5, 0.5)$

6.4 Decoupled model

As can be seen in 16, the first four states travel, travel rate, pitch and pitch rate are completely decoupled from elevation and elevation rate. One of the reasons that might be the reason as to why the system still works is that the first states are mostly reliant on difference in actuation between the motors, while the last two are dependent on the sum of equal thrust in each of the motors. When the pitch of the helicopter is completely horizontal, the elevator has no way of regaining its height, except for the collisions we saw in 16 where the helicopter model gently bounced into a relatively more horizontal position.

1. The first suggested solution A to the decoupling problem is to rewrite the equation of motion for $\ddot{e}$ into a coupling between pitch and elevation through $C_p V_s \cos(p) + C_e \cos(e)$ [1]. The rewritten equation properly depicts the dependency between pitch and elevation, especially in extreme cases like the one where $p = \frac{\pi}{2}$. As a result, this seems like a good solution to the problem.

2. The second suggestion B implements a joint to the objective function in $+\sum_{k+1}^{N}(\dot{e}_k - \dot{e}_{k-1} - \Delta t C_p V_s \cos(p_k) + \Delta t C_e \cos(e_k))^2$ [1]. This joint tries to penalize deviations instead of introducing the coupling itself. This solution might work in most cases, but will probably struggle in cases as the one mentioned in point one. In addition, introducing such an extensive calculation into the objective function might introduce unnecessary computation, while not managing to handle the coupling as wanted. Accordingly, this seems like a poor solution.

3. The final suggestion C wants to re-choose the weights of the matrix Q for each time step during the flight based on the pitch angle [1]. By dynamically adjusting the elevation rate weight based on $\cos(p)$, the model can become aware of dependancies between pitch and elevation without the direct coupling that suggestion A provides. This suggestion results in a reasonably good solution, but would be even better if introduced in combination with A.

6.5 MATLAB and Simulink

6.5.1 Code from section 6

```
"../init_files_2021_v2/init.m";

delta_t = 0.25; % sampling time
N = 40;

K_6_5 = -K_3*K_ep
K_6_6 = -K_3* K_ed

A = [
    0, 1,       0,          0,          0,          0;
    0, 0,     -K_2,         0,          0,          0;
    0, 0,       0,          1,          0,          0;
    0, 0,   -K_1*K_pp, -K_1*K_pd,       0,          0;
```

```
    0, 0,       0,          0,          0,          1;
    0, 0,       0,          0,       K_6_5,     K_6_6
];

B = [
    0,       0;
    0,       0;
    0,       0;
 K_1*K_pp,   0;
    0,       0;
    0,     K_3*K_ep
];

% Initial values
x1_0 = pi;                                  % Lambda
x2_0 = 0;                                  % r
x3_0 = 0;                                  % p
x4_0 = 0;                                  % p_dot
x5_0 = 0;                                  % e
x6_0 = 0;                                  %e_dot

% Initial values
x0 = [x1_0 x2_0 x3_0 x4_0, x5_0, x6_0]';
% Discrete time system matrices
A_disc = eye(6) + delta_t * A;
B_disc = delta_t * B;

nx = size(A,2);
nu = size(B,2);

% Generate system matrixes for linear model
Aeq = gen_aeq(A_disc, B_disc, N, nx, nu);
beq = zeros(size(Aeq,1),1);
beq(1:nx) = A_disc*x0;

q1 = 2;
q2 = 5;

% Bounds
ul = 0;                 % Lower bound on control
uu = (1/6) * pi;        % Upper bound on control

xl = -Inf*ones(nx,1); % Lower bound on states (no bound)
xu = Inf*ones(nx,1);  % Upper bound on states (no bound)
xl(3)= ul;            % Lower bound on state x3
```

```
xu(3)= uu;              % Upper bound on state x3

% Generate constraints on measurements and inputs
[vlb,vub]        = gen_constraints(N,N,xl,xu,ul,uu);
% We want the last input to be zero
vlb(N*nx+N*nu) = 0;
vub(N*nx+N*nu) = 0;

alpha = 0.2;
beta = 20;
lambda_t = (2* pi) / 3;

A_eq = 0;

Q = diag([2,0,0,0,0,0]);
R = diag([q1, q2]);
G = blkdiag( kron(eye(N),Q), kron( eye(N), R));
C = zeros(size(G,1), 1);

% Initialize z for the whole horizon
z  = zeros(N*nx+N*nu,1);
z0 = z;

f = @(z) 1/2 * z' * G * z;
opt = optimoptions('fmincon', 'Algorithm', 'sqp', 'MaxFunEvals',
   40000);
[Z,ZVAL,EXITFLG] = fmincon(f, z0, [],[], Aeq, beq, vlb, vub, @
   constraints,opt);

%Q_lqr = diag([5,.2,.1,15,30,10]);
Q_lqr = diag([5,.3,.1,15,30,10]);
R_lqr = diag([.5,.5]);
disp(R_lqr)
[K,S,P] = dlqr(A_disc, B_disc, Q_lqr, R_lqr);
disp(K)

u1  = [Z(N*nx+1:nu:N*nx+N*nu);Z(N*nx+N*nu-1)]; % Control input
   from solution
u2  = [Z(N*nx+2:nu:N*nx+N*nu);Z(N*nx+N*nu)];

%The different states retrieved from generated solution
x1 = [x0(1);Z(1:nx:N*nx)];
x2 = [x0(2);Z(2:nx:N*nx)];
x3 = [x0(3);Z(3:nx:N*nx)];
x4 = [x0(4);Z(4:nx:N*nx)];
```

```
x5 = [x0(5);Z(5:nx:N*nx)];
x6 = [x0(6);Z(6:nx:N*nx)];



num_variables = 5/delta_t;
zero_padding = zeros(num_variables,1);
unit_padding  = ones(num_variables,1);

u1  = [zero_padding; u1; zero_padding];
u2  = [zero_padding; u2; zero_padding];
x1  = [pi*unit_padding; x1; zero_padding];
x2  = [zero_padding; x2; zero_padding];
x3  = [zero_padding; x3; zero_padding];
x4  = [zero_padding; x4; zero_padding];
x5  = [zero_padding; x5; zero_padding];
x6  = [zero_padding; x6; zero_padding];

optTraj = [x1, x2, x3, x4, x5, x6];

t = 0:delta_t:delta_t*(length(u1)-1);
optTraj = timeseries(optTraj,t);
paadrag1 = timeseries(u1, t);
paadrag2 = timeseries(u2, t);

function [c,c_eq] = constraints(z)
global N alpha beta lambda_t nx
   c = zeros(N,1);
   for k = 1:N
       c(k) = alpha * exp(-beta*(z(1+(k-1)*nx)-lambda_t)^2) - z
          (5+(k-1)*nx);
   end
   c_eq = [];
end
```

6.6 Simulink block diagrams

6.6.1 Simulink from section 6

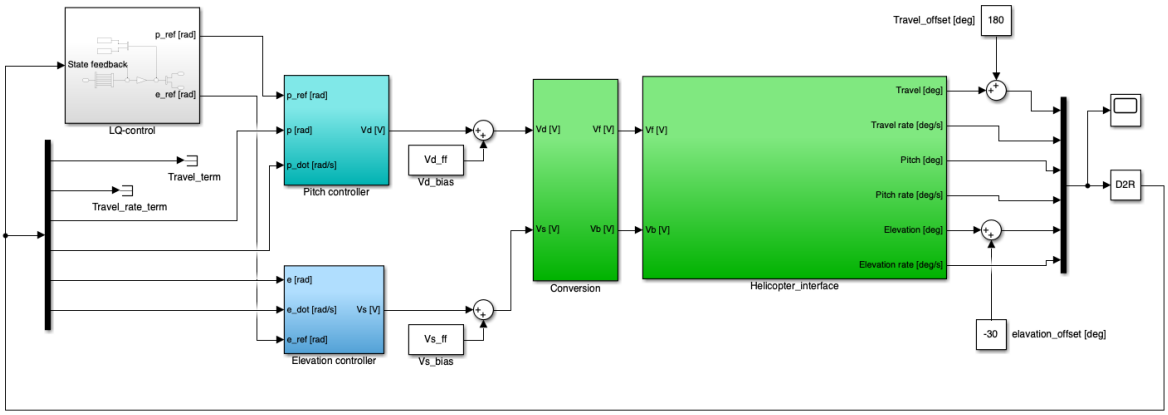


Figure 19: Final block diagram

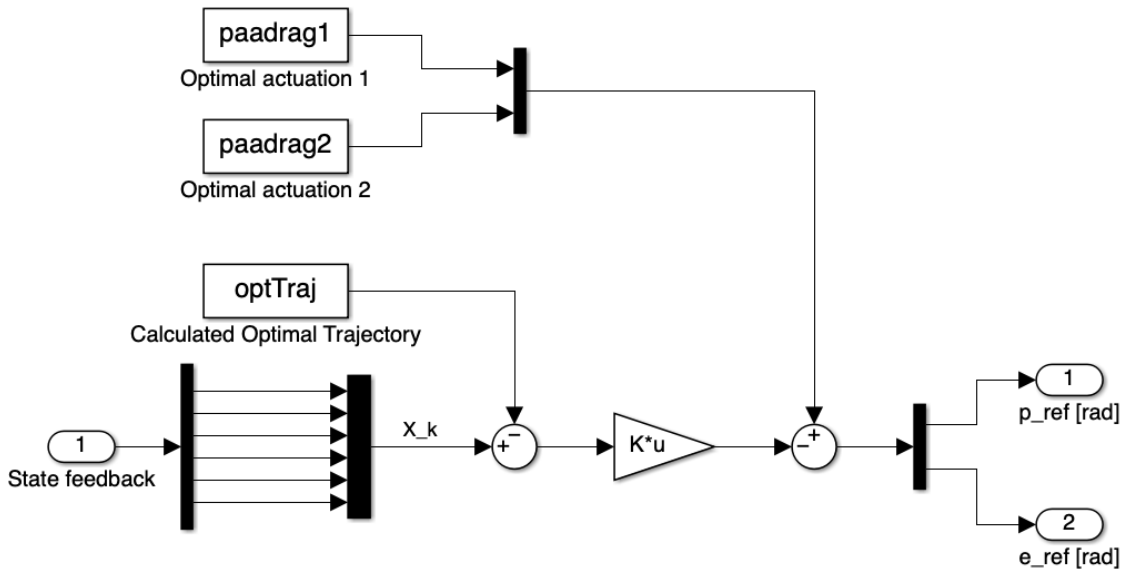


Figure 20: LQR controller

7 Reflection

When working with this lab, we could not fathom why the helicopter introduced an overshoot in travel for all our testing. None of the tuning efforts managed to mitigate this behavior, so we asked ourselves; there must be some error in the code. And indeed it was. Now on the day of the edited project delivery, a major mistake in our system showed itself:

The input constraints ul and uu were set to the values of 0 and 1 respectively. What this meant for the system, was that for correcting the travel rate after reaching the goal position the helicopter was not "allowed" to use a negative input causing the system to wait until the entire system prediction had passed before changing course The single bane of our solution were these two lines of code that followed with the template code for every section.

The actual values of these constraints should have been set to $ul = -\frac{pi}{6}$ and $uu = \frac{pi}{6}$.

```
% Bounds
ul          = -pi/6;                    % Lower bound on control
uu          = pi/6;                    % Upper bound on control
```

Just like predicted the helicopter showed marvelous behavior with the optimal tuning found through our experiments shown in Figure 21:

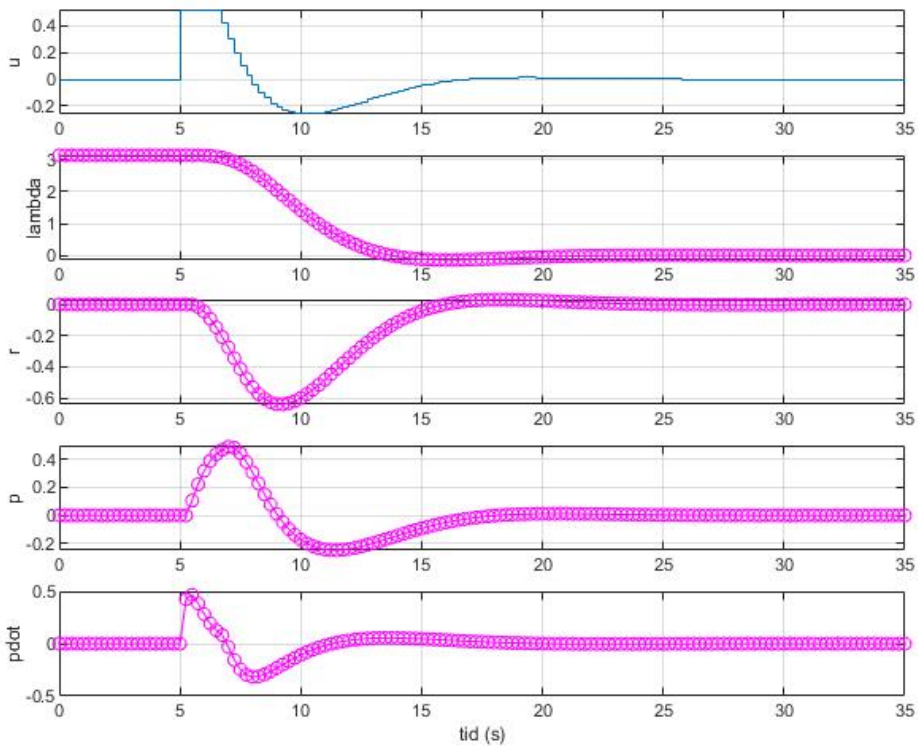


Figure 21: Fmincon prediction post-bound-correction.

When it came to actual data collection with this newly made correction, time constraints led to our decicion in only testing it for the current optimal iteration we had. That being the tuning Q = diag(5, 0.3, 0.1, 15, 30, 10) and R = diag(0.5, 0.5). The actual results of testing the helicopter with this implementation is shown in Figure 22:

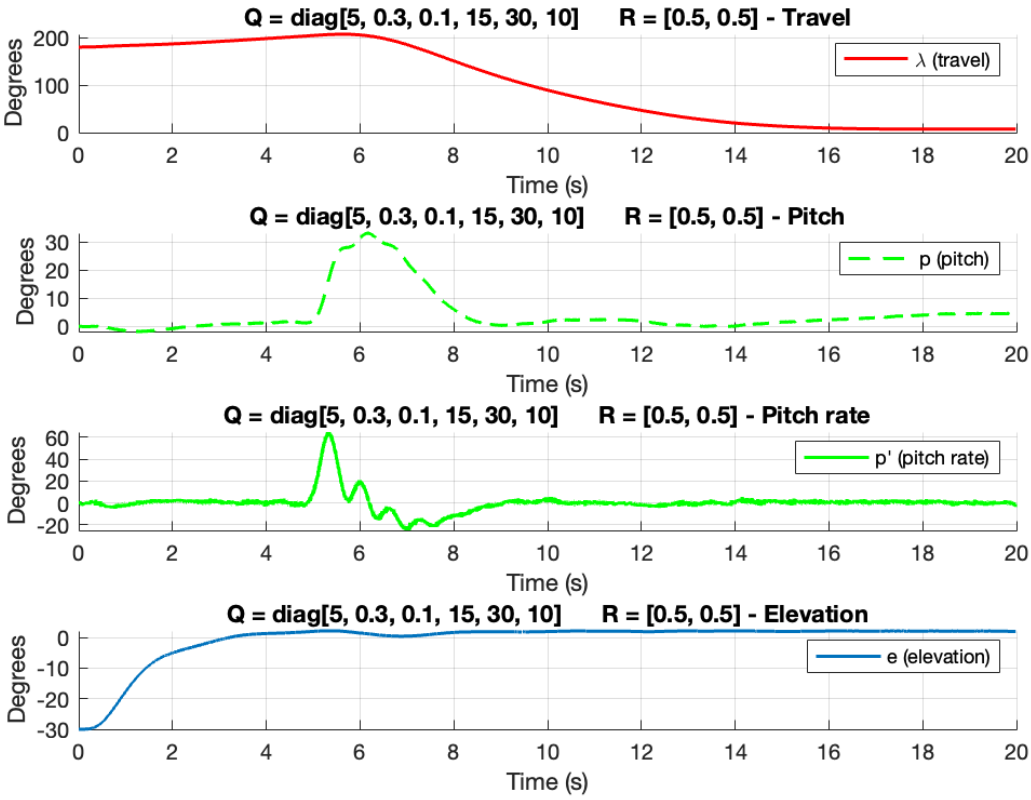


Figure 22: Fmincon results post-bound-correction.

What can be seen here in 22 is night and day in comparison to the previous results seen in 6 just from this minuscule change. As mentioned the time constraints on the revision of this project led to us only running this single simulation and seeing the results, you could tell we were ecstatic.

Although, we think that our procedural findings in the previous iteration of this project got to show that we made justified decisions when tuning and further improving the helicopter model behavior and in the reflecting upon the results we got. Because of this we did not re-gather data for every aspect of the project and instead focused on altering the report based on the feedback provided by the research assistant Tore.

References

[1] Department of Engineering Cybernetics. *TTK4135 Optimization and Control Helicopter Lab*. NTNU - Norwegian University of Science and technology, Updated: Jan. 2025.